CAR RENTAL SYSTEM

Course Code: CP317

Group ID: TEAM #28

Team Members and Roles:

- SabilUllah Hussaini (Product Owner)
- Sahil Sandhu (Product Owner & Developer)
- Moosa Ahmed (Scrum Master)
- Mahad Farrukh (Developer)
- Ayesha Raheel (Developer)
- Kritika Kamath (Developer)
- Shalin Panjwani (Developer)

FINAL PROJECT DESCRIPTION

Project Summary

Car Rental System is an inclusive digital solution intended to revolutionize the method of vehicle rentals for customers and administrators of the rental company alike. The system will automate this process by finding available vehicles, making reservations, keeping rental history, and maintaining an updated inventory. It addresses inefficiencies in traditional rental workflows with a responsive, user-friendly interface and a secure backend powered by C# and SQLite.

Purpose:
The system is intended to offer seamless, end-to-end vehicle rentals. Consequently, customers can easily find cars with flexible filters, view their detailed specifications, add them to a shopping cart, and finally make reservations by simulating the process of making a payment. Administrators will have an integrated portal for managing users, vehicles, bookings, and business performance.

Target Audience:
Customers seeking to hire vehicles easily and securely.
Administrators of rental companies who manage fleets, oversee users, and carry out business analytics.

Final Delivered Features:
Secure user registration, login, and profile setup.
Dynamic search of vehicles with filters: province, city, dates, make, model, year, type, price range
Vehicle results including detailed information and images
Shopping cart system with persistent storage
Complete reservation workflow with simulated payment
Customer dashboard showing upcoming and past bookings
Admin portal with user CRUD, car CRUD, and booking management
KPI dashboard: Revenue, Fleet Utilization, Bookings & More
Consistent, accessible UI with error handling and validation

FINAL PRODUCT BACKLOG

Subsection A:

The following user stories were fully completed during Sprints 1–3. Each story includes its ID, title, priority, story points, and assigned sprint.

Story ID	Title	Priority	Points	Sprint	Status
AUTH-1	Secure Account Creation	High	3	Sprint 1	DONE
AUTH-2	Secure Login & Logout	High	2	Sprint 1	DONE
UI-1	Vehicle Search	High	3	Sprint 1	DONE
UI-2	Search Results - Basic Info	High	5	Sprint 1	DONE
UI-3	Vehicle Details Page	High	5	Sprint 2	DONE
UI-4	Filter Search by Category/Price	Medium	5	Sprint 2	DONE
USER-1	User Dashboard	Medium	5	Sprint 2	DONE
RES-1	Shopping Cart	High	3	Sprint 2	DONE
RES-2	Reservation & Payment	High	5	Sprint 2	DONE
ADMIN-1	Admin Login Portal	High	3	Sprint 3	DONE
ADMIN-2	Vehicle Inventory Management	High	8	Sprint 3	DONE
PAY-1	Payment Integration	High	5	Sprint 3	DONE
REP-1	Admin Dashboard &	Medium	8	Sprint 3	DONE

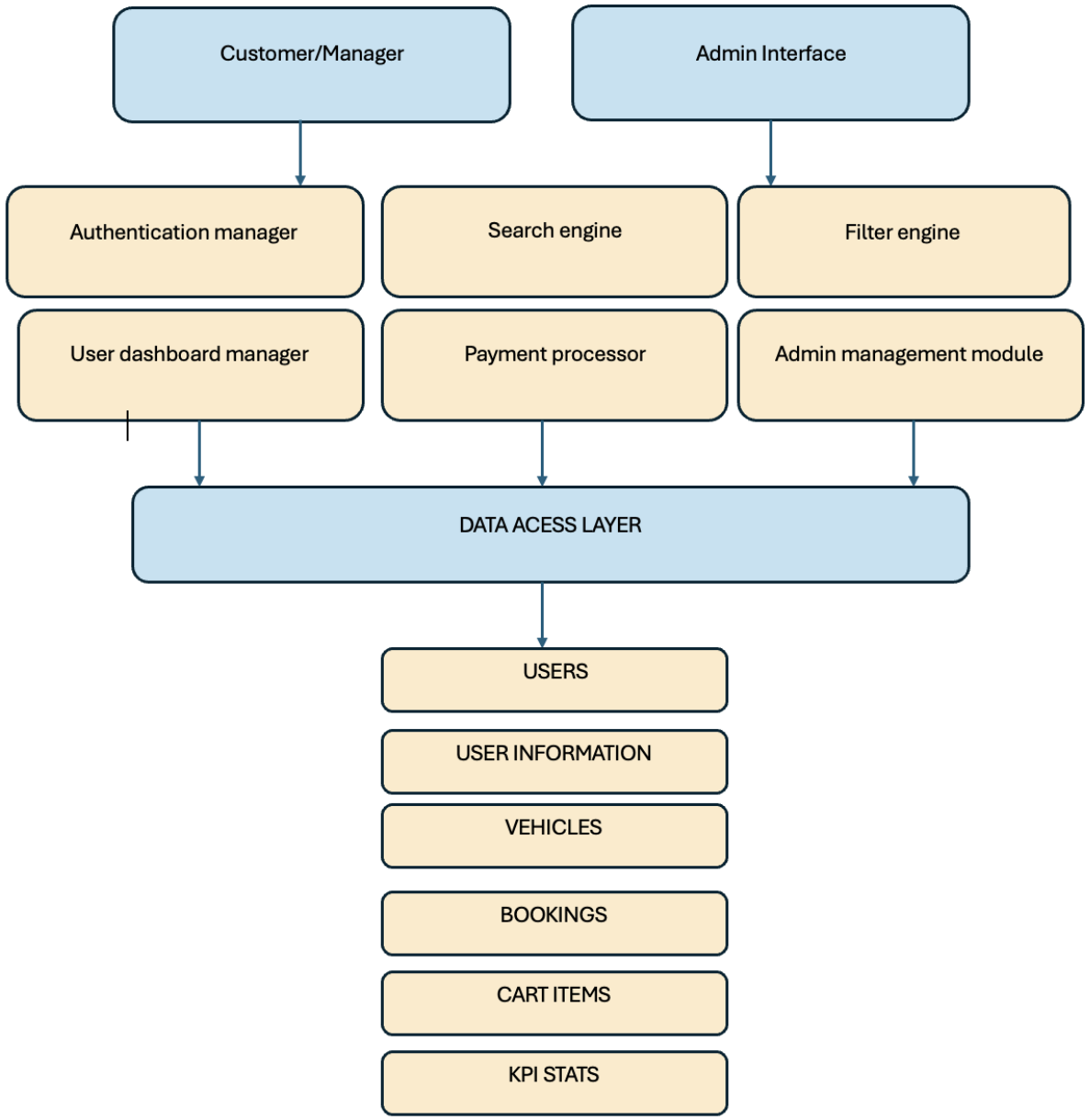
Subsection B:

There are no remaining or deferred stories. All planned backlog items were completed in Sprints 1-3.

Subsection C:

The complete product backlog is provided in the attached excel file.

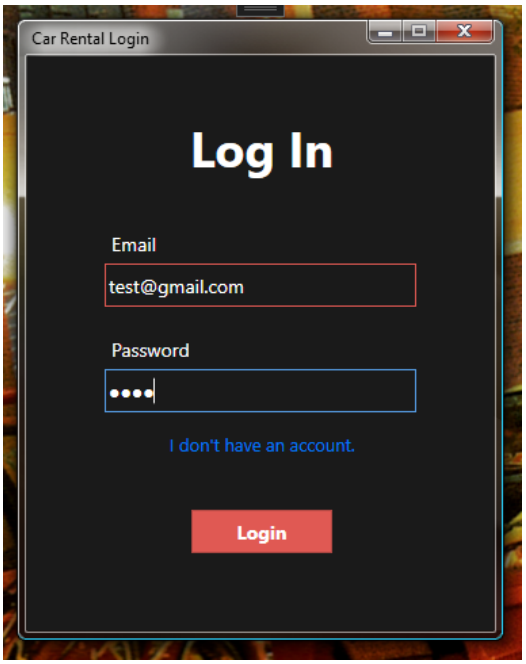
Design



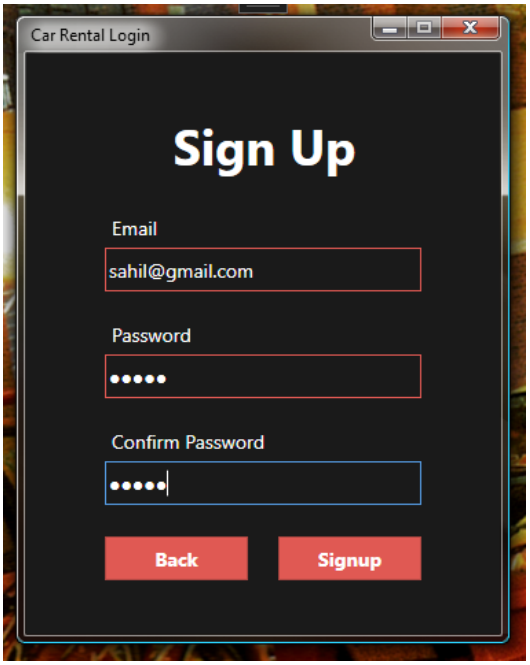
The Car Rental System uses a layered architecture to separate the user interface, core logic, and data storage.
The Presentation Layer (WPF UI) contains the Customer Manager and Admin Interface, which handle all user interactions.
The Business Logic Layer includes modules such as the Authentication Manager, Search Engine, Filter Engine, Booking Manager, Payment Processor, and Admin Management Module. These components process requests and enforce system rules.
The Data Access Layer (Entity Framework) manages communication between the logic and the database.

The Database Layer (SQLite) stores all system data, including Users, UserInformation, Vehicles, Bookings, CartItems, and KPI Stats.

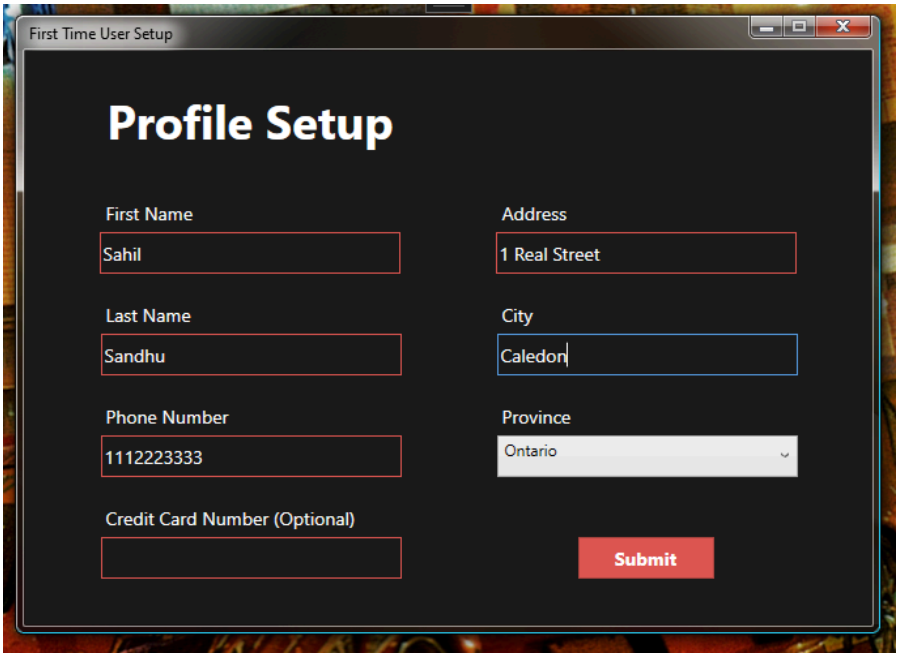
Login Screen



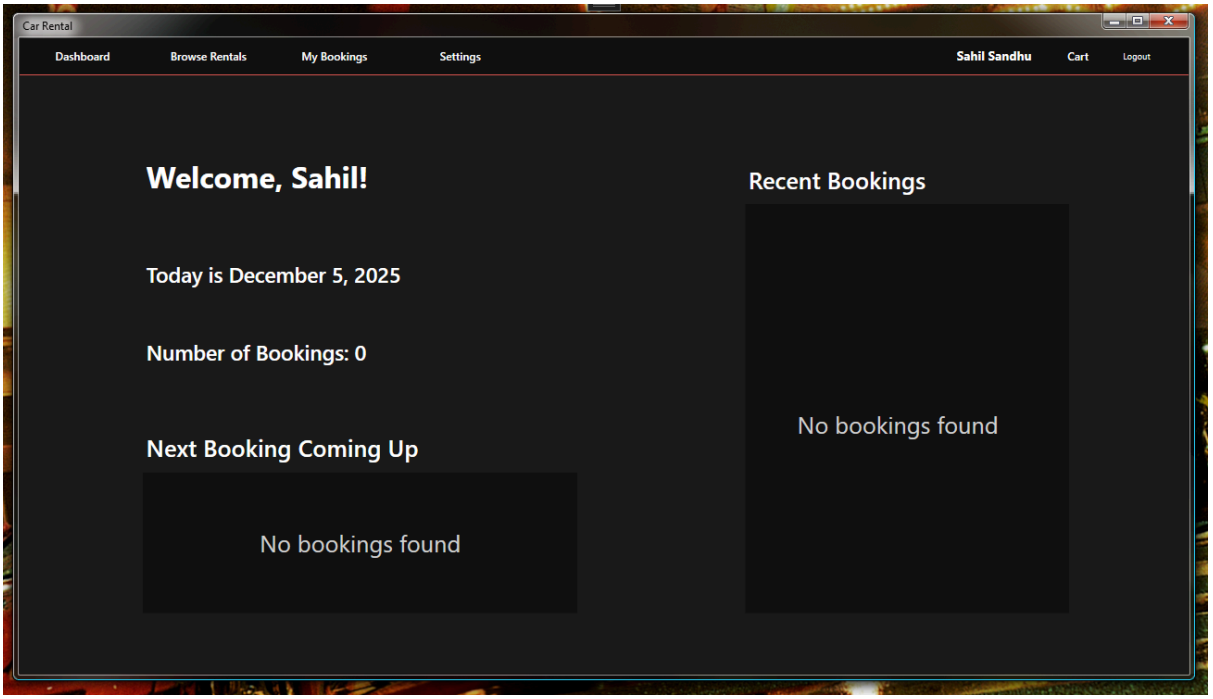
Signup Screen



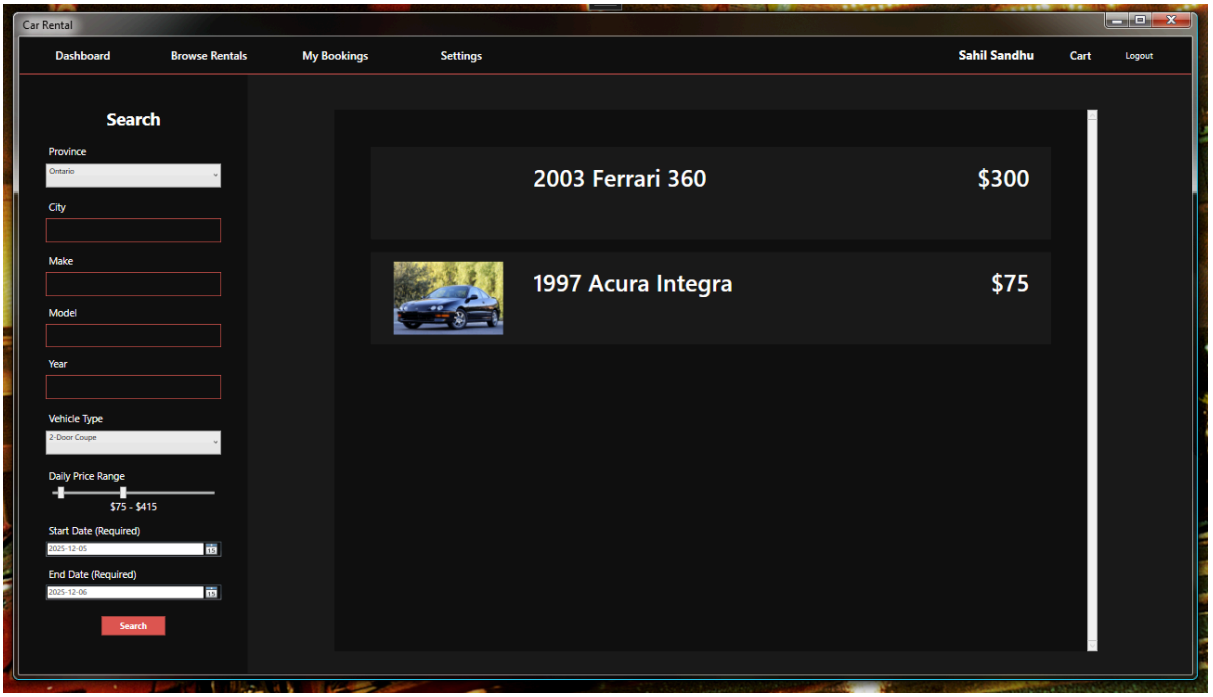
First Time User Setup (Only shown after an account is created. The account cannot be logged in until this is filled out.)



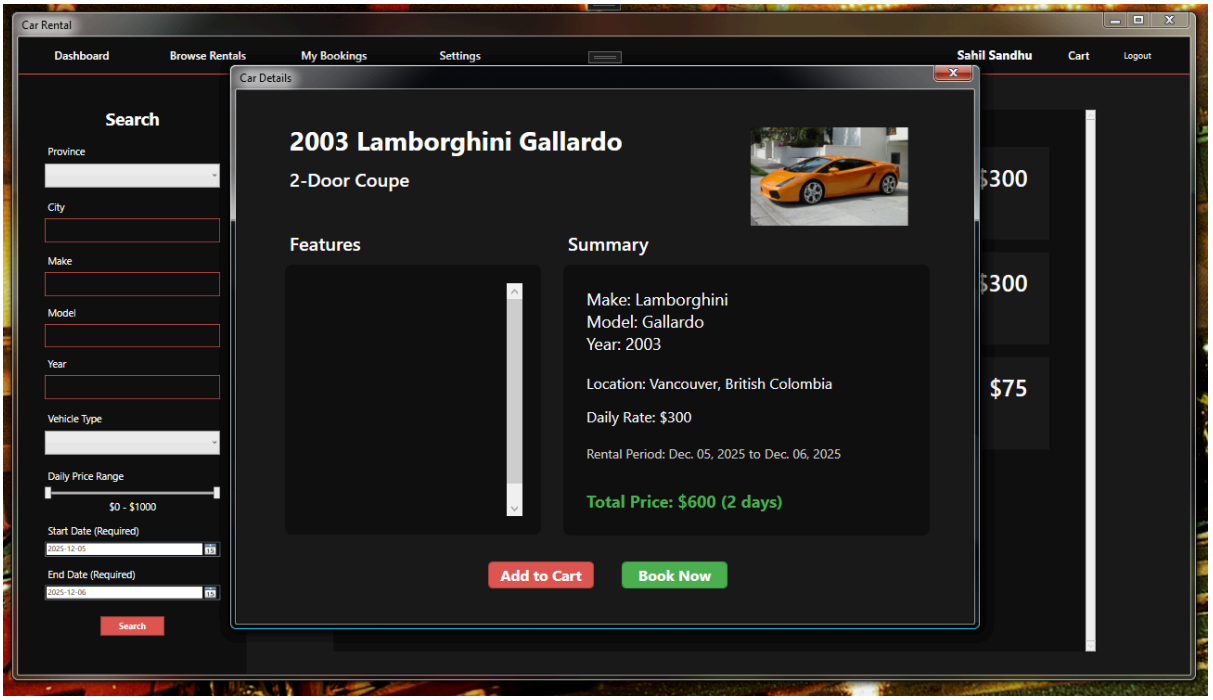
Dashboard



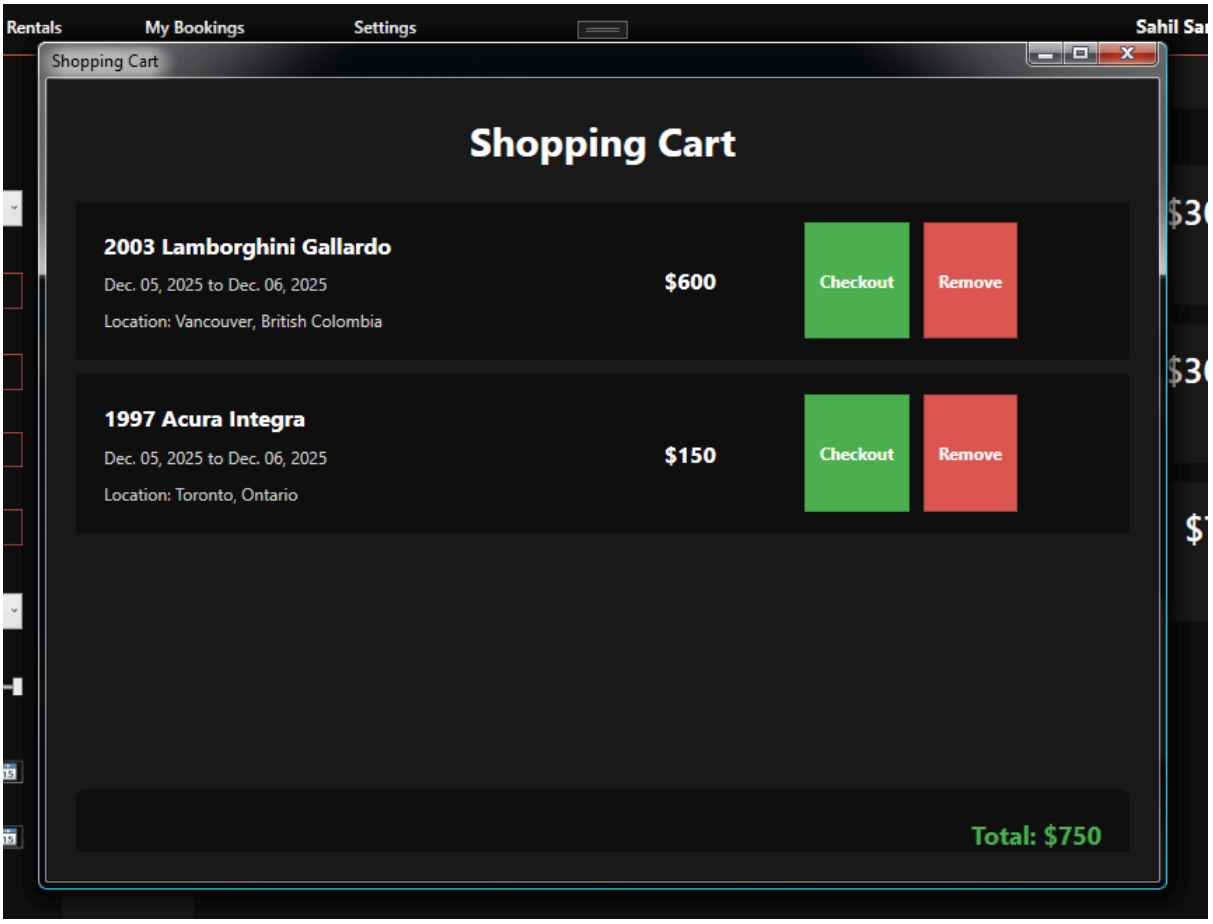
Rental Browser



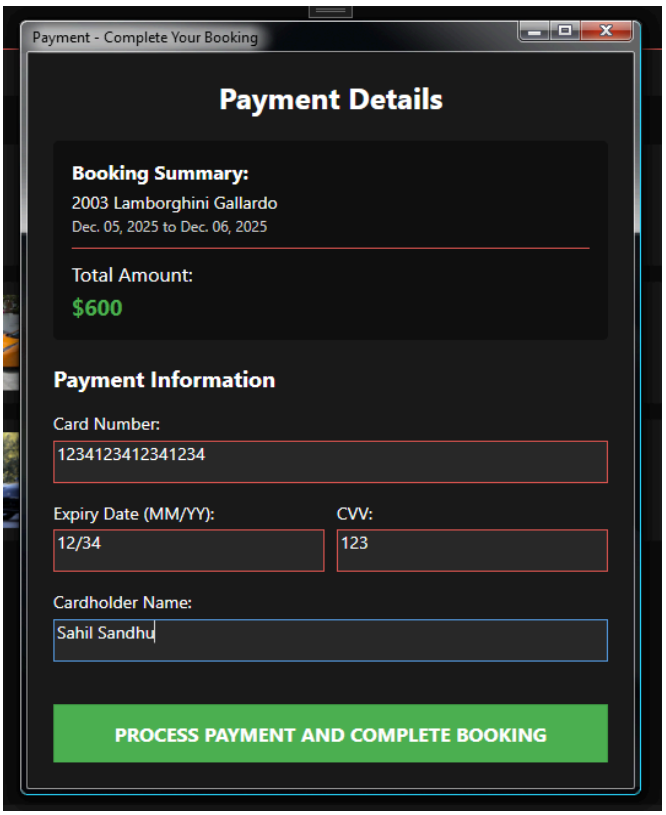
Car Details Screen



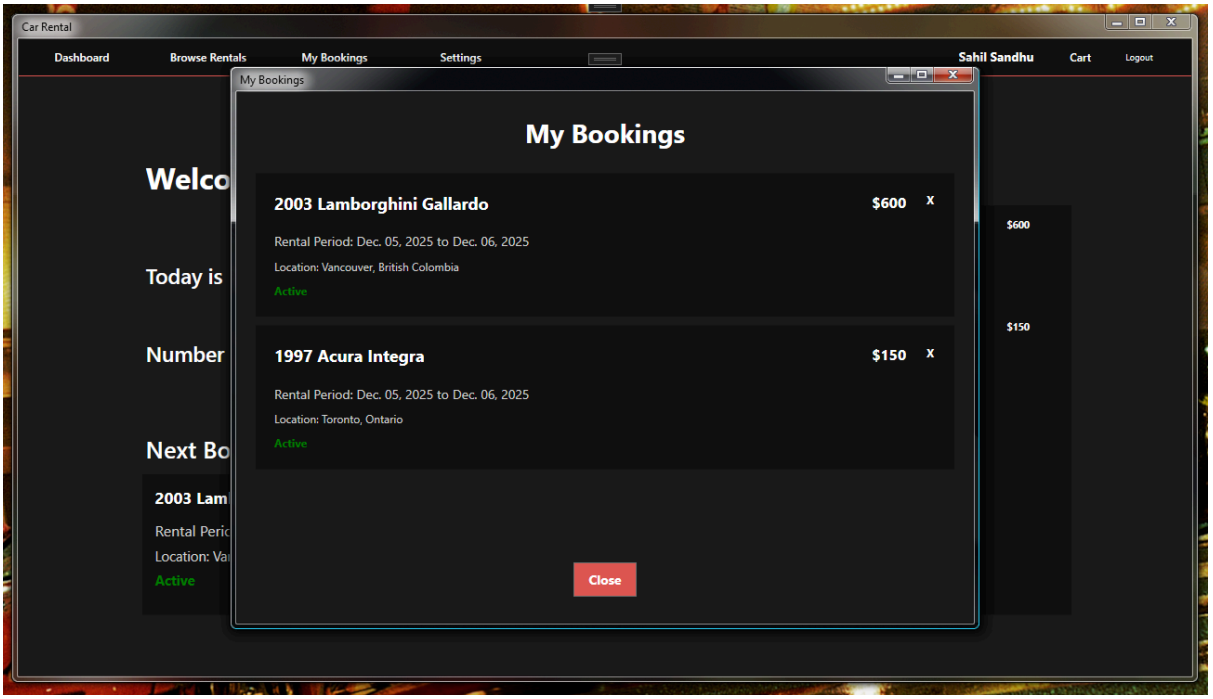
Shopping Cart



Checkout Screen

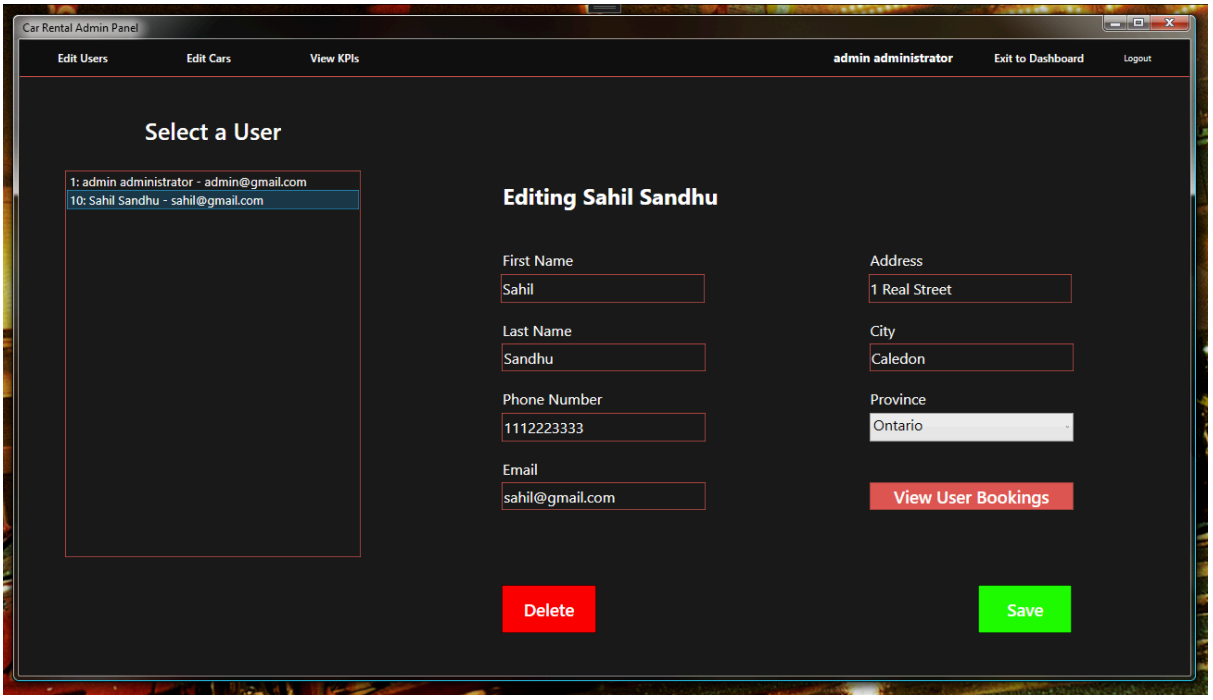


My Bookings Screen

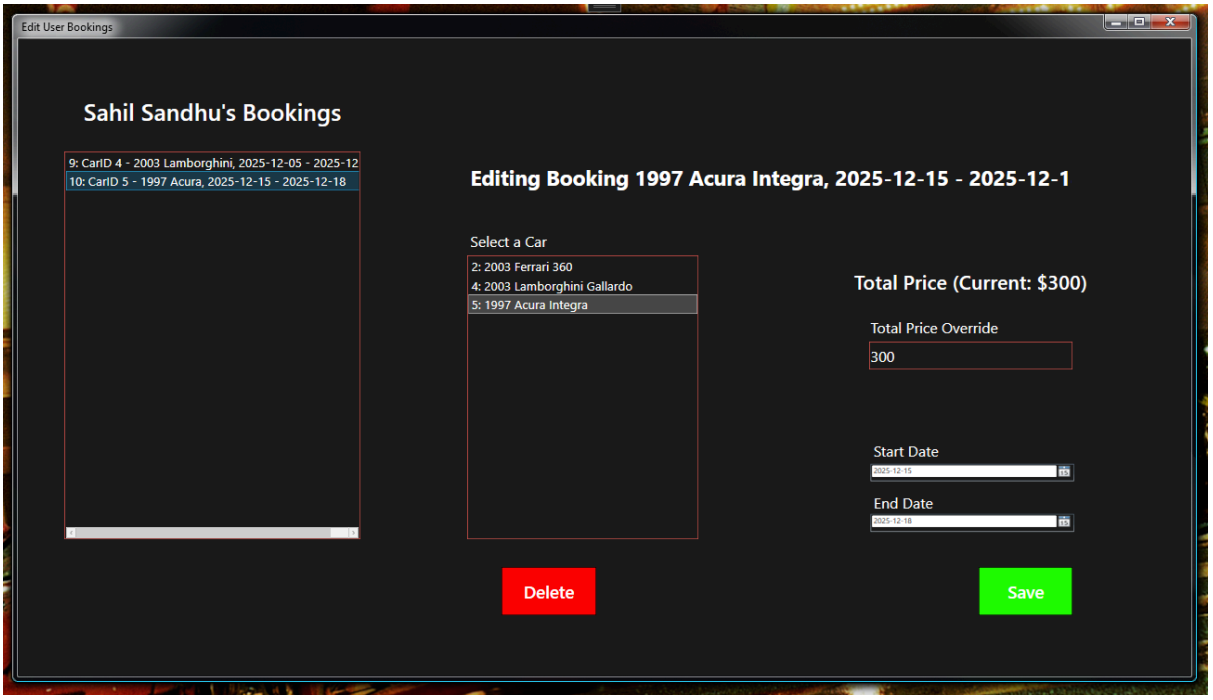


Everything below is part of the Admin Panel.

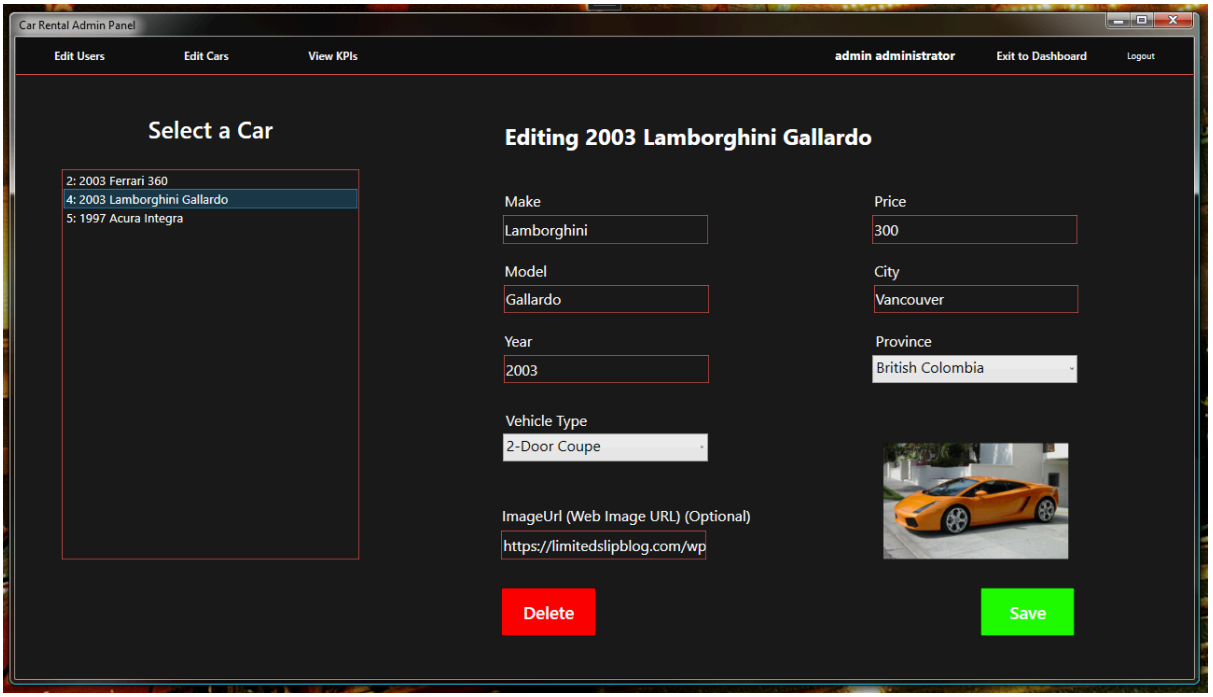
Edit Users Screen



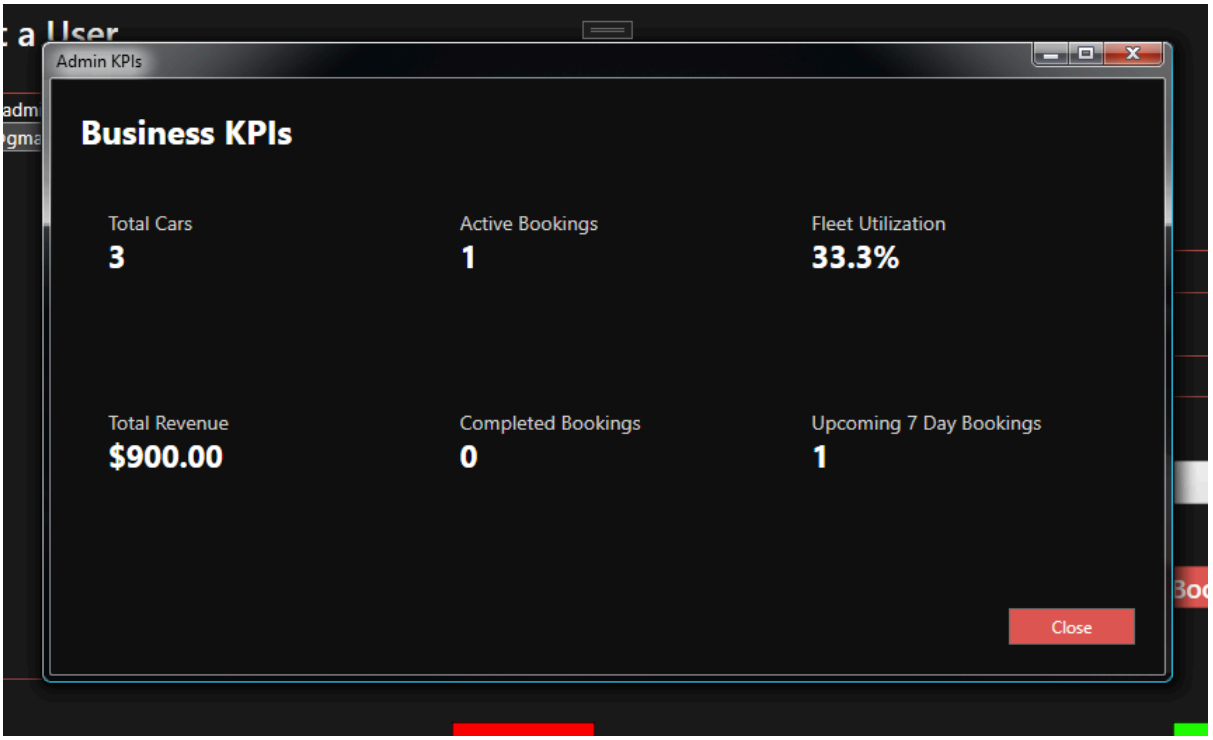
Edit User Bookings Screen



Edit Cars Screen


Car Rental Admin Panel
Edit Users
Edit Cars
View KPIs
admin administrator
Exit to Dashboard
Logout
Select a Car
2: 2003 Ferrari 360
4: 2003 Lamborghini Gallardo
5: 1997 Acura Integra
Editing 2003 Lamborghini Gallardo
Make
Lamborghini
Model
Gallardo
Year
2003
Vehicle Type
2-Door Coupe
Price
300
City
Vancouver
Province
British Colombia
ImageUrl (Web Image URL) (Optional)
https://limitedslipblog.com/wp
Delete
Save

KPI Viewer


Admin KPIs
Business KPIs
Total Cars
3
Active Bookings
1
Fleet Utilization
33.3%
Total Revenue
$900.00
Completed Bookings
0
Upcoming 7 Day Bookings
1
Close

Technical Decisions
When we first began work on this project, we were using SQLite, but we soon chose to move to PostgreSQL as our SQL management framework as it allows for multiple connections, as well as having many more variable types compared to SQLite, a big one for our program being DateTime. Since the start of development we have been using EFCore to write our SQL code, as we find it simpler to write and read compared to pure SQL queries. EFCore also makes it easier to manage the database with things like migrations. On the user interface side, we chose to use WPF as it has an in depth visual editor which is also easy to use.

USER MANUAL

This section explains how to install, configure, and run the Car Rental System. It provides everything the evaluator needs to launch the executable, connect to the database, and log into both a regular user and an admin account.

System Requirements

Required Software:

1. .NET Runtime 9.0
    - The installer runs automatically when opening the CarRental executable if .NET 9 is missing.
    - Optional manual download: https://dotnet.microsoft.com/en-us/download/dotnet/9.0
2. PostgreSQL & pgAdmin 4
    - Download link:
    - https://www.enterprisedb.com/downloads/postgres-postgresql-downloads

Test Credentials

These accounts are included in the database backup and will work immediately after importing the SQL file.

Admin Account

- Email: admin@gmail.com
- Password: admin

Regular User Account

- Email: test@gmail.com
- Password: cp317test

If the evaluator wants, they may also create new accounts through the Sign-Up window in the application.

Initial Setup

A. Extract the Provided Files

From the final submission package, extract:

- /CarRental_CP317.exe
- /full_server_backup.sql
- Any additional supporting files included by the team

B. Import the PostgreSQL Database

Open Command Prompt and run the following command:

- "C:\<PostgreSQL Location>\PostgreSQL\<Version>\bin\psql.exe" -U postgres -f "<Extracted Folder>\full_server_backup.sql"

If PostgreSQL is already added to your PATH, you may use the shorter version:

- psql -U postgres -f "<Extracted Folder>\full_server_backup.sql"

This creates the full CarRental database schema, including:

- Users
- Cars
- Bookings
- Shopping Cart
- Admin accounts

No manual table creation is required.

C. (Optional) Connect via pgAdmin 4

If the evaluator wants to visually inspect the tables:

1. Open pgAdmin 4
2. Create a Server with the following details:
    - Name: CarRental_CP317
    - Host: localhost
    - Password: your local PostgreSQL password
3. pgAdmin will automatically find the imported CarRental database and load all tables.

Your system's connection string expects the PostgreSQL password that the evaluator uses on their machine. They only need to make sure their local PostgreSQL credentials match what they type during installation.

Running the Application

1. Double-click CarRental_CP317.exe
2. If .NET 9.0 is not installed, the program will prompt an automatic installer
3. After launching, the first window is the Login Screen

From here, the evaluator may either:

Option 1 — Log in using test credentials

Use the credentials provided.

Option 2 — Create a new account

Click Sign Up, enter the required information, and the system will automatically store the new user in the PostgreSQL database.

Navigating the System

After logging in, the user gains access to the complete system:

A. Home & Search

- Enter pickup and return dates
- Enter a city and province
- Press Search to view available cars

B. Vehicle Listings

Listings show:

- Model
- Make
- Year
- Price
- Location
- Image

Users can select a car to view full details.

C. Car Details

From here, the user can:

- View full specs
- See total cost for selected dates
- Add the car to their cart
- Or click Book Now for instant reservation

D. Shopping Cart

Shows all cars added by the user along with:

- Dates
- Price
- Location

Users can remove vehicles or check out individually.

E. Payment Window

Users enter simulated payment details:

- Card number
- Expiry
- CVV
- Cardholder name

The system validates the fields and then:

- Creates a booking
- Updates the car's booking history
- Removes the item from the cart (if applicable)

F. My Bookings

Shows all active and completed reservations.

G. Admin Dashboard (Admin Account Only)

After logging in as admin, the system displays additional options.

Closing the Application

Simply close any window using the close button. The system correctly disposes database connections on exit.

SUMMARY TESTING

Testing Methodology

Testing for the Car Rental System was carried out through four main approaches:

Unit Testing

Individual components such as vehicle search logic, booking price calculations, user input validation, payment form validation, and admin CRUD operations were tested in isolation to confirm that each feature behaved as intended before system integration.

Integration Testing

Modules were tested together to ensure data consistency between the UI, business logic, and PostgreSQL database. Examples include:

- Login → Dashboard loading
- Search filters → Search results → Vehicle details
- Booking creation → Cart → Payment → Database update
- Admin edits → User-side updates (car listings, pricing, bookings

System-Level Testing

End-to-end workflows across the entire application were tested, including the full user journey from sign-up to reservation confirmation, as well as all administrative operations. This verified that the complete system functions as a unified product.

Manual Functional Testing

Testers manually interacted with the built application (.exe), performing realistic user actions such as browsing vehicles, creating accounts, checking out, and updating inventory. This helped catch UI-related and flow-related issues that automated test styles might miss.

Story ID	Test Performed	Expected Result	Actual Result	Status
AUTH-1	Create user account with valid inputs	Account is created and stored in database	Works as expected	**PASS**
AUTH-2	Login and logout with valid credentials	User session starts and ends correctly	Works as expected	**PASS**

UI-1	Search cars using date and location	Available cars load correctly	Correct results displayed	**PASS**
UI-2	View basic information in search results	Vehicle details (make, model, year, price) display correctly	Matches database values	**PASS**
UI-3	Open vehicle details page	Full vehicle info and rental period pricing appear	Works as expected	**PASS**
UI-4	Apply category/price filters	Results update instantly with correct matching cars	Filters work correctly	**PASS**
USER-1	Load user dashboard	Shows current and past bookings correctly	Works as expected	**PASS**
RES-1	Add car to shopping cart	Item appears in cart and persists in database	Works as expected	**PASS**
RES-2	Complete reservation workflow	Booking is created and cart updates accordingly	End-to-end booking works	**PASS**
ADMIN-1	Admin login portal	Only admin-flagged accounts gain access	Correct behaviour observed	**PASS**
ADMIN-2	Admin vehicle inventory management	Add/edit/delete vehicles update system immediately	User-side search reflects changes	**PASS**
PAY-1	Simulated payment processing	Valid card input produces confirmed booking; invalid input is rejected	Works as expected	**PASS**

REP-1	Admin KPI dashboard	KPIs display accurate real-time data from SQL database	Metrics update correctly	**PASS**

Known Issues & Limitations

1. No Real Payment Gateway
    - The payment module is fully simulated. It performs input validation but does not connect to a real payment processor. No actual financial transactions occur.

2. Minor UI Alignment Variations
    - On certain display resolutions, some elements may shift slightly. These do not affect functionality.

3. Admin Flag Must Be Set in Database
    - Admin accounts can only be created or flagged through SQL/pgAdmin for security reasons.

Ethical dimensions report

Privacy risks, Security considerations, Mitigations

The system stores personal information such as login credentials and booking details, creating privacy risks if accessed by unauthorized users. To mitigate this, passwords are hashed, admin functions are restricted, and no real payment data is stored because the checkout process is simulated. Security is further supported through validation on all forms. Future improvements could include encrypting stored data and adding audit logs.

Accessibility and Fairness/Bias Considerations

The interface uses simple layouts, readable text, and clear navigation to support basic accessibility. Although not fully WCAG compliant, the design minimizes confusion and supports a broad range of users. Fairness is maintained by ensuring that search results, prices, and booking rules are based only on rental criteria (dates, vehicle type, location) and never on personal or demographic information.

Reflection: Implemented vs. Improvements

Implemented:

- Password hashing and restricted admin access
- Simple and consistent UI for ease of use
- Equal treatment of all users with unbiased search and booking logic

Could be improved:

- Add encryption for stored personal data
- Implement stronger accessibility features
- Add detailed logging and more advanced security controls

Final Reflection

What is fully complete?

All planned features were completed, including user authentication, search and filtering, vehicle details, shopping cart, booking and payment simulation, user dashboard, full admin management tools, and KPI analytics.

What was the biggest technical challenge?

Integrating all the modules into a single working system, especially ensuring correct booking logic, data consistency, and smooth interaction between the customer interface, admin interface, and database. What would you improve if you had more time? We would improve the UI design, enhance its accessibility features, implement real payment processing, enhance the security aspects with encryption and audit logs, and optimize for performance to ensure a seamless user experience.

What would you improve with more time?

We would enhance the UI design, add stronger accessibility features, implement real payment processing, improve security through encryption and audit logs, and optimize performance for a smoother user experience.

DEMO VIDEO:
https://drive.google.com/file/d/1iLMCQsNMiHS6YndLuD7_6bwm3g6KA07g/view?usp=sharing

ABOUT THE PROJECT:

https://wilfrid-laurier.zoom.us/rec/share/vzUMhpLKIq4KWx859XCibvSNUatdpBbmEe-09pXYepq1faGQJSKAqgbycyjWA9Ya.VZDFWiO5zyXkuXYm?startTime=1764997036000
Passcode: u#5q.tfm